EARLY ACCESS / CORE V1 SPECIFICATION EDITION

Learning STARK

Safe Systems Programming for Typed ML Deployment

CHAPTER 04

Control Flow and Pattern Matching

STARK LANGUAGE PROJECT / JULY 2026

Learning STARK

Chapter 4 preview: *Control Flow and Pattern Matching*

Early Access manuscript - July 2026

This chapter documents a language specification under active implementation. Core v1 is a normative draft and has not yet been validated by a conforming compiler.

Source manuscript: `book/manuscript/ch04-control-flow-pattern-matching.md`

CHAPTER 04

04

Control Flow and Pattern Matching

Control flow becomes dependable when every path has a type, every exit has a destination, and every state has a case.

Programs become interesting when they choose, repeat, and stop. Those actions look familiar in STARK: `if`, `loop`, `while`, `for`, `break`, `continue`, `return`, and `match` all have recognizable syntax. The important difference is not their spelling. It is the way they participate in the type system.

An `if` can produce a value. A `match` must account for every possible input. A plain `loop` can finish by carrying a value through `break`. A `for` loop consumes an iterator rather than relying on a special numeric-loop rule. An enum makes alternative states explicit, including the data carried by each alternative. `Option<T>` uses those same enum and pattern rules to represent absence without introducing null into every reference type.

These rules turn control flow into more than a sequence of jumps. The compiler can ask whether every branch agrees on a result type, whether every enum variant is handled, whether an exit is legal at its location, and whether code can ever be reached. That analysis gives later ownership and error-handling chapters a dependable foundation.

This chapter uses Core v1 syntax and semantics. The examples are specification-valid illustrations; the compiler remains under implementation.

> **Specification status**
>
> Core v1 defines the grammar and typing rules described here. The current Rust compiler scaffold does not yet provide a conforming implementation of all control-flow, exhaustiveness, and reachability checks. Diagnostics shown in this chapter are illustrative unless explicitly quoted as a required error code.

4.1 Control flow is typed structure

Many languages separate expressions, which produce values, from statements, which perform actions. STARK keeps the distinction but lets several control-flow forms be expressions:

- blocks;
- `if` expressions;
- `match` expressions; and
- `loop` expressions.

`while` and `for` are also block-formed expressions, but their type is always `Unit`. They are normally used for their effects. A block receives the type of its trailing expression, or `Unit` when it has no trailing expression.

STARK

```
// Core v1
let temperature = 72;
let label = if temperature > 80 {
    "hot"
} else if temperature < 50 {
    "cold"
} else {
    "comfortable"
};
```

The entire `if` has type `&str`. Each reachable result branch has that type. The semicolon belongs to the surrounding `let` statement, not to the final expression inside a branch.

This expression orientation reduces temporary mutable state. A statement-oriented version might declare `label`, assign it in three branches, and depend on a separate definite-initialization analysis. The expression-oriented version makes the relationship direct: choose exactly one value, then bind it.

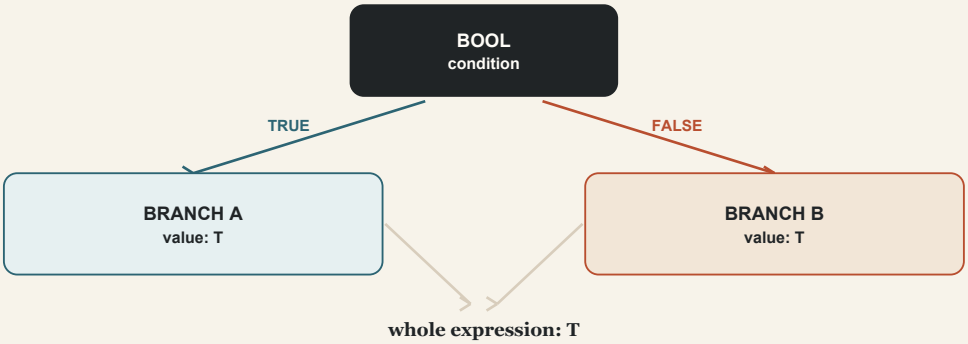

Figure 4-1. Typed control flow selects one path while preserving one result contract.

Typing control flow requires the compiler to combine local facts. It checks the condition, checks every branch in its own lexical scope, finds a common result type, and then assigns that type to the complete expression. If no valid common type exists, compilation fails before any path is executed.

4.2 `if` selects one value

An `if` condition must have type `Bool`. STARK has no truthiness conversion for integers, strings, collections, or references.

```stark
// Core v1
let retry_count = 2u32;

if retry_count > 0u32 {
    println("retry enabled");
}

// if retry_count { } // Error: expected Bool, found UInt32.
```

The explicit comparison documents the decision. It also avoids language-specific questions such as whether negative numbers, empty strings, or empty collections count as true.

An `if` with `else` must unify

When an `if` contains an `else`, every branch must produce one type. Identical types unify directly:

STARK

```
// Core v1
fn batch_size(low_memory: Bool) -> UInt32 {
    if low_memory {
        8u32
    } else {
        32u32
    }
}
```

The final `if` is also the function body's trailing expression, so its `UInt32` value becomes the return value.

Numeric types do not implicitly widen to make branches agree:

STARK

```
// Core v1
// let size = if compact { 8u16 } else { 32u32 };
// Error: branch types UInt16 and UInt32 do not unify.

let size = if compact {
    8u16 as UInt32
} else {
    32u32
};
```

The cast makes the representation change visible at the branch where it occurs.

The never type `!` can coerce to any type. This permits a branch that cannot return normally to coexist with a value-producing branch:

STARK

```
// Core v1
let workers = if configured_workers > 0u32 {
    configured_workers
} else {
    panic("worker count must be positive")
};
```

The success branch has type `UInt32`; `panic` returns `!`. The complete `if` has type `UInt32` because the failing branch never supplies a competing value.

An `if` without `else` is `Unit`

Omitting `else` means the condition may be false and no branch body will run. Core v1 therefore gives the expression type `Unit`, and the body must also produce `Unit`.

```stark
// Core v1
if cache_hit {
    println("using cached result");
}
```

The call is used as a statement and the block has no trailing value. This is valid.

```stark
// Core v1
// let result = if cache_hit { 42 };
// Error: an if without else has type Unit;
// the branch must also have type Unit.
```

If a value is required, make the alternative explicit with `else`, or model optional production with `Option<T>`.

Parsing block-formed expressions

A block-formed expression may stand as a statement without a semicolon:

```stark
if ready {
    start();
}
println("continued");
```

Inside a block, statement parsing is greedy. To use a block-formed value as an operand before more syntax, parenthesize it:

```stark
let adjusted = (if fast { 10 } else { 20 }) - 1;
```

At the very end of a block, an un-terminated block-formed expression is the block's trailing value. This is why a function can return an `if` directly.

4.3 `loop` repeats until an explicit exit

The plain `loop` form has no condition. It repeats its body until control leaves through `break`, `return`, a non-returning call, or another enclosing control transfer.

```stark
// Core v1
let mut attempts = 0u32;

loop {
    attempts += 1u32;
    if attempts == 3u32 {
        break;
    }
}
```

A bare `break;` exits the nearest loop with `Unit`. When every reachable `break` is bare, the `loop` expression has type `Unit`.

Unlike `while` and `for`, a plain `loop` may produce a value. Attach the value to `break`:

```stark
// Core v1
let selected = loop {
    let candidate = next_candidate();
    if is_acceptable(&candidate) {
        break candidate;
    }
};
```

The type of `selected` is the unified type of all `break` operands in that loop. Every value-carrying exit must agree.

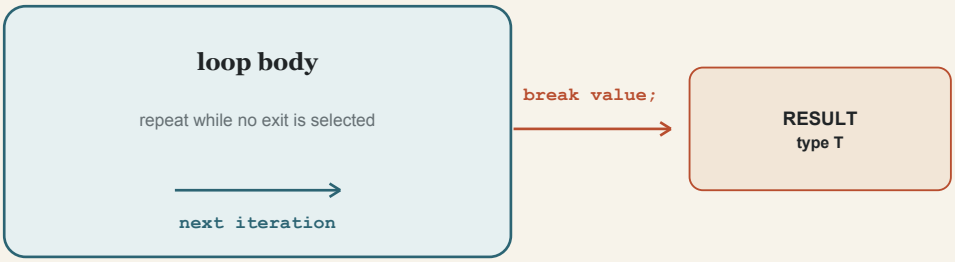

Figure 4-2. A plain loop can return a value through a typed break edge.

```stark
// Core v1
let code = loop {
    if first_condition() {
        break 10u32;
    }
    if second_condition() {
        break 20u32;
    }
};
```

Both exits carry `UInt32`, so the loop has type `UInt32`. Mixing a bare break with a value break would fail to unify `Unit` with the value type.

A `loop` with no reachable `break` never completes normally and has type `!`:

```stark
// Core v1
fn service_forever() -> ! {
    loop {
        serve_next_request();
    }
}
```

This fact matters to surrounding flow analysis. Code after an unconditional call to `service_forever` is unreachable.

4.4 `while` repeats while a Boolean holds

A `while` loop checks its `Bool` condition before each iteration:

```stark
// Core v1
let mut remaining = 3u32;

while remaining > 0u32 {
    println(remaining.fmt().as_str());
    remaining -= 1u32;
}
```

If the condition begins false, the body runs zero times. A `while` expression always has type `Unit`, and `break` inside it must not carry a value.

```stark
// Core v1
while connection_is_open() {
    if shutdown_requested() {
        break;
    }
    process_message();
}
```

Use `while` when continuation depends on state observed before each pass. Use a value-producing `loop` when the exit itself discovers the result. A `while` loop can still update an outer mutable binding, but that design should be deliberate; a value-producing `loop` often expresses the data flow more clearly.

The condition is evaluated again after every iteration that reaches the bottom of the body or executes `continue`. Side effects in the condition are possible when function calls are involved, but a simple named predicate is usually easier to review.

4.5 `for` consumes an iterator

STARK's `for` syntax is:

```stark
for item in expression {
    // body
}
```

The expression must have a type implementing `Iterator`. On every iteration, the loop binding receives the iterator's `Item` type. Integer ranges implement `Iterator`, making counted traversal concise:

```stark
// Core v1
for index in 0u32..4u32 {
    println(index.fmt().as_str());
}
```

The half-open range yields `0`, `1`, `2`, and `3`. An inclusive range `0u32..=3u32` yields the same values while expressing a different boundary model.

Collections and slices expose iteration through library methods:

```stark
// Core v1
fn print_scores(scores: &[Int32]) {
    for score in scores.iter() {
        println(score.fmt().as_str());
    }
}
```

The precise `Item` type comes from the iterator implementation. An iterator may yield owned elements, shared references, mutable references, or another defined item type. Reading the iterator signature is therefore part of reasoning about ownership, not merely about repetition.

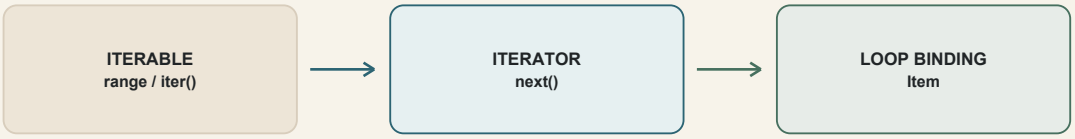

Figure 4-3. A for loop receives successive Item values from an Iterator.

A `for` loop always has type `Unit`. A break inside it must be bare:

STARK

```
// Core v1
for candidate in candidates.iter() {
    if candidate.is_ready() {
        break;
    }
}
```

When the loop needs to produce a selected value, update an outer `Option<T>`, call a library operation such as `find` when its ownership behavior fits, or use a plain value-producing `loop` over an iterator explicitly. Each form communicates a different contract.

4.6 `break`, `continue`, and `return` have destinations

Control-transfer statements are legal only where they have a destination.

`break` exits the nearest enclosing loop. `continue` skips the remainder of the current iteration and starts the next iteration of the nearest loop. Both are errors outside a loop.

STARK

```
// Core v1
for value in values.iter() {
    if should_ignore(value) {
        continue;
    }
    if should_stop(value) {
        break;
    }
    process(value);
}
```

Core v1 does not define loop labels, so `break` and `continue` always target the innermost loop. In nested traversal, extracting the inner operation into a function or representing the outcome as an enum is often clearer than trying to simulate a multi-level jump.

`return` exits the current function. It may carry a value compatible with the declared return type, or be bare in a `Unit`-returning function:

STARK

```
// Core v1
fn first_positive(values: &[Int32]) -> Option<Int32> {
    for value in values.iter() {
        if *value > 0 {
            return Option::Some(*value);
        }
    }
    Option::None
}
```

The explicit early return handles the successful search. Falling through to the trailing expression produces `None`.

Code after an unconditional transfer cannot execute:

STARK

```
fn answer() -> Int32 {
    return 42;
    // println("unreachable"); // Warning W0005.
}
```

Core v1 classifies unreachable code as warning `W0005`, while a missing return on some path is an error. A function with a non-`Unit` return type must produce a compatible value on every normal path.

4.7 Enums define closed alternatives

A struct combines fields that exist together. An enum declares a closed set of alternatives, called variants. Each value has exactly one variant at a time.

STARK

```
// Core v1
enum DeploymentState {
    Queued,
    Loading { model: String },
    Ready { workers: UInt32 },
    Failed(String)
}
```

This declaration uses all three variant forms:

- `Queued` is a unit variant with no payload;
- `Loading` and `Ready` are struct variants with named fields; and
- `Failed` is a tuple variant with a positional payload.

The type is `DeploymentState`, not the individual variant name. Constructing a value chooses one alternative:

```stark
let initial = DeploymentState::Queued;
let active = DeploymentState::Ready { workers: 4u32 };
let failed = DeploymentState::Failed(String::from("model rejected"));
```

Because the alternatives are closed, the compiler knows the complete set. That knowledge powers exhaustive matching. Adding a new variant is a meaningful schema change: matches that enumerate all existing variants must be reviewed.

Enums are sometimes called sum types because the possible values come from one variant or another. Structs are product types because every field participates in the value. The terminology matters less than the design distinction: use a struct for simultaneous facts and an enum for mutually exclusive states.

4.8 Patterns inspect and bind structure

A `match` evaluates one scrutinee expression, compares it with arms from top to bottom, and evaluates the expression belonging to the selected pattern.

```stark
// Core v1
fn state_label(state: DeploymentState) -> &str {
    match state {
        DeploymentState::Queued => "queued",
        DeploymentState::Loading { model: _ } => "loading",
        DeploymentState::Ready { workers: _ } => "ready",
        DeploymentState::Failed(_) => "failed"
    }
}
```

All four arms produce `&str`, so the match expression has type `&str`. Match arms may include an optional trailing comma. Consistent commas are usually easier to maintain.

Patterns can test values, ignore components, and introduce bindings. Core v1 includes:

- literal patterns such as `0`, `'x'`, or `true`;
- `_`, the wildcard pattern;
- identifier bindings;
- unit, tuple, and struct enum-variant patterns;
- tuple patterns; and
- fixed-length array patterns.

Bindings extract payloads

STARK

```
// Core v1
fn describe(state: DeploymentState) -> String {
    match state {
        DeploymentState::Queued => String::from("waiting"),
        DeploymentState::Loading { model } => model,
        DeploymentState::Ready { workers } => workers.fmt(),
        DeploymentState::Failed(message) => message
    }
}
```

The shorthand `{ model }` means `{ model: model }`: match the named field and bind it to a local variable with the same name. `Failed(message)` binds the positional payload.

Moving or borrowing through patterns follows the ordinary ownership rules. This chapter focuses on pattern structure; Chapters 7 and 8 examine when extracting a non-`Copy` field moves it and how matching through references affects access.

Tuple and array patterns

STARK

```
// Core v1
let point = (12, 8);
let quadrant = match point {
    (0, 0) => "origin",
    (0, _) => "vertical axis",
    (_, 0) => "horizontal axis",
    (_, _) => "off axis"
};
```

STARK

```
// Core v1
let marker: [UInt8; 3] = [83u8, 84u8, 75u8];
let recognized = match marker {
    [83u8, 84u8, 75u8] => true,
    [_, _, _] => false
};
```

An array pattern has a fixed number of elements in Core v1. The pattern must be compatible with the scrutinee type.

Identifier patterns require attention

A single identifier pattern that resolves to a unit enum variant or constant matches that value. Otherwise it introduces a new binding, which matches anything.

Multi-segment paths such as `DeploymentState::Queued` always match by value. Qualified variant names make production matches easier to read and reduce the chance that a name intended as a constant is interpreted as a catch-all binding.

4.9 Exhaustiveness turns omissions into errors

A match must cover every possible value of its scrutinee type. For an enum, listing every variant is exhaustive. A wildcard arm also covers whatever earlier arms did not.

STARK

```
// Core v1
enum Health {
    Healthy,
    Degraded,
    Unavailable
}

fn should_route(health: Health) -> Bool {
    match health {
        Health::Healthy => true,
        Health::Degraded => true
        // Error E0303: non-exhaustive match; Unavailable is missing.
    }
}
```

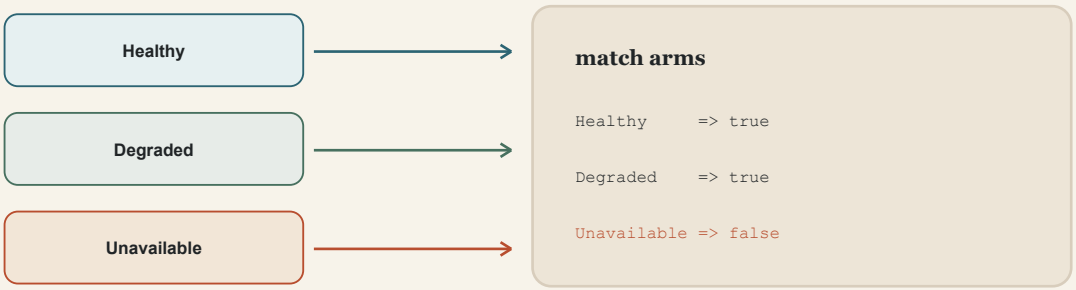

Figure 4-4. Exhaustiveness requires every variant to reach an arm or a catch-all.

Adding the missing variant gives the decision an explicit policy:

```stark
match health {
    Health::Healthy => true,
    Health::Degraded => true,
    Health::Unavailable => false
}
```

Alternatively, a wildcard can express a genuine default:

```stark
match status_code {
    200 => "ok",
    404 => "not found",
    _ => "other"
}
```

Literal patterns over unbounded integer or string domains cannot enumerate every possible value, so a catch-all binding or wildcard is normally required. Finite domains can be covered explicitly. For `Bool`, the arms `true` and `false` are exhaustive.

Tuple exhaustiveness depends on the coverage of each position. Enum payload patterns must also account for the payload's relevant possibilities when nested patterns distinguish them.

Arm order and unreachable patterns

Arms are attempted in source order. A wildcard or unconstrained binding matches every remaining value, so later arms cannot be selected:

```stark
match code {
    _ => "any code",
    200 => "success" // unreachable pattern
}
```

The normative specification requires exhaustiveness. Full match-usefulness warnings are deliberately deferred in the current implementation plan, so authors should not depend on the early compiler diagnosing every redundant arm. Put specific cases before broad ones and treat arm order as part of the reviewable policy.

All arm result expressions must unify, just like `if` branches:

```stark
let message = match state {
    DeploymentState::Queued => "waiting",
    DeploymentState::Ready { workers: _ } => "running",
    DeploymentState::Loading { model: _ } => "loading",
    DeploymentState::Failed(_) => panic("deployment failed")
};
```

The first three arms return `&str`; the last has type `!`, which coerces to `&str`. Therefore `message` has type `&str` on every path that completes normally.

4.10 `Option<T>` models absence without null

Core v1 does not make every reference or object nullable. Absence is represented by the standard enum `Option<T>`:

```stark
enum Option<T> {
    Some(T),
    None
}
```

`Some(value)` carries a `T`. `None` carries no value. A caller cannot use the contained `T` without first accounting for which variant it received.

```stark
// Core v1
fn lookup_port(name: &str) -> Option<UInt16> {
    if name == "http" {
        Option::Some(80u16)
    } else if name == "https" {
        Option::Some(443u16)
    } else {
        Option::None
    }
}
```

The function's signature states that absence is a normal outcome. The caller handles it with `match`:

```stark
let port = match lookup_port("https") {
    Option::Some(value) => value,
    Option::None => 8080u16
};
```

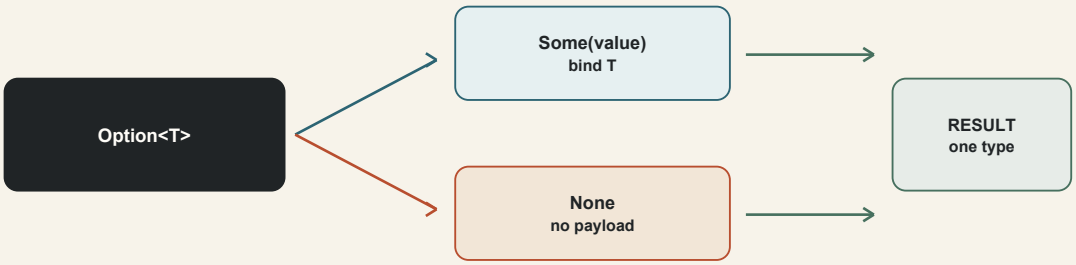

Figure 4-5. Option makes both presence and absence explicit in the value and the control flow.

The type of `port` is `UInt16`; both arms produce that type. There is no unchecked dereference and no sentinel value such as zero that might also be a legitimate domain value.

The standard library also defines methods including `is_some`, `is_none`, `unwrap`, `unwrap_or`, `map`, and `and_then`. `unwrap` is convenient only when the program's invariant genuinely proves the value is present; otherwise an explicit match makes the missing case visible. Chapter 6 returns to `Option` alongside `Result` and the `?` operator.

Optional production versus conditional effects

An `if` without `else` is appropriate for an optional effect because its result is `Unit`:

```stark
if verbose {
    println("starting request");
}
```

When a computation may or may not produce a value, return `Option<T>`:

```stark
fn positive(value: Int32) -> Option<Int32> {
    if value > 0 {
        Option::Some(value)
    } else {
        Option::None
    }
}
```

The difference is semantic. One form conditionally performs work; the other communicates a two-case data result.

4.11 Case study: a deployment state transition

Consider a controller that receives events while preparing a model deployment. An enum can make both the current state and the incoming event closed, inspectable sets.

```stark
// Core v1
enum Phase {
    Idle,
    Loading { model: String },
    Serving { model: String, workers: UInt32 },
    Failed { message: String }
}

enum Event {
    Load(String),
    Loaded { workers: UInt32 },
    Rejected(String),
    Stop
}
```

The transition function takes ownership of the current phase and event, then returns the next phase:

STARK

```
// Core v1
fn transition(phase: Phase, event: Event) -> Phase {
    match (phase, event) {
        (Phase::Idle, Event::Load(model)) => {
            Phase::Loading { model }
        },

        (Phase::Loading { model }, Event::Loaded { workers }) => {
            if workers > 0u32 {
                Phase::Serving { model, workers }
            } else {
                Phase::Failed {
                    message: String::from("worker count is zero")
                }
            }
        },

        (Phase::Loading { model: _ }, Event::Rejected(message)) => {
            Phase::Failed { message }
        },

        (Phase::Serving { model: _, workers: _ }, Event::Stop) => {
            Phase::Idle
        },

        (current, _) => current
    }
}
```

The scrutinee is a tuple, so each arm can examine the combination. Specific legal transitions come first. The final `(current, _)` arm preserves the phase for all other events. It is the explicit default policy.

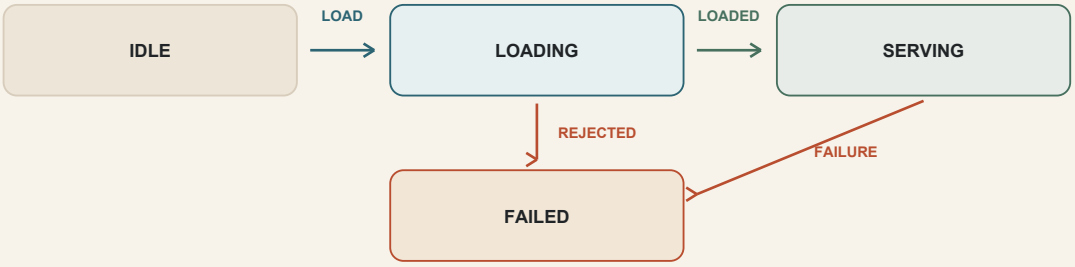

Figure 4-6. Events choose typed transitions; the fallback policy preserves the current phase.

Every arm returns `Phase`, including the nested `if`. The compiler therefore assigns `Phase` to the match and to the function's trailing expression.

The final catch-all makes the match exhaustive but also means a newly added event will initially be ignored in states not covered by new specific arms. That may be correct, or it may hide an incomplete transition policy. Exhaustiveness proves that a value is handled; it does not prove that the chosen behavior is the domain behavior you intended.

For a strict controller, represent invalid combinations explicitly:

```stark
enum TransitionResult {
    Applied(Phase),
    Ignored(Phase),
    Invalid { phase: Phase, event: Event }
}
```

That design forces the caller to decide what to do with rejected transitions. The best enum is not the one with the fewest variants. It is the one that makes operationally distinct cases impossible to confuse.

4.12 Read control flow as a proof

When reviewing a function, trace control flow in a repeatable order.

1. **Identify each expression's required type.** Start with the function return type, annotated bindings, and operator operands.
2. **Check conditions.** Every `if` and `while` condition must be `Bool`.

3. **Classify each loop.** Decide whether it is a value-producing `loop` or a `Unit`-producing `while` or `for`.
4. **Locate every transfer.** Connect each `break` and `continue` to the nearest loop and each `return` to the current function.
5. **Unify result paths.** `if` branches, `match` arms, and value-carrying `break` operands must agree.
6. **Check coverage.** A match must cover the scrutinee's entire value space.
7. **Check reachability.** Code after unconditional `return`, `break`, or a `!` expression cannot execute in that path.
8. **Review defaults as policy.** A wildcard proves coverage but may conceal a new case that deserves explicit behavior.

For the `transition` case study, the outer requirement is `Phase`. The tuple match covers all combinations because the final arm matches any current phase and any remaining event. Each specific arm produces `Phase`. The `Loaded` arm contains an `if` whose two branches also produce `Phase`. No break or continue is present, and the match is the function's trailing expression. The control-flow proof closes.

This way of reading scales. In a small function it catches a missing `else`; in a production state machine it reveals where a generic fallback may be too permissive.

Chapter summary

- Control-flow forms participate in static typing; they are not unstructured jumps.
- An `if` condition must be `Bool`. With `else`, all branches unify to one type.
- An `if` without `else` has type `Unit`, and its body must also produce `Unit`.
- A plain `loop` may produce the unified value carried by `break value;`.
- A `loop` with no break has type `!`; `while` and `for` always have type `Unit`.
- `for` requires an `Iterator` and binds each successive `Item` value.
- `break` and `continue` target the nearest loop; Core v1 has no loop labels.
- Every normal path through a non-`Unit` function must produce the declared return type.
- Enums define a closed set of mutually exclusive variants, optionally carrying data.
- Patterns test values, destructure payloads, ignore components, and introduce bindings.
- Match arms run in source order, must unify to one result type, and must be exhaustive.
- A wildcard supplies coverage, but its broad policy deserves deliberate review.
- `Option<T>` represents presence or absence without making every value nullable.

Exercises

1. What is the type of `if ready { 1u32 } else { 2u32 }`? What changes if the `else` branch returns `2u64`?
2. Explain why `if count { process(); }` is invalid when `count` is `UInt32`. Write the explicit condition.
3. Rewrite a mutable `result` binding assigned inside two branches as a single value-producing `if`.
4. Determine the type of a `loop` with two exits: `break 4u16;` and `break 9u16;`. What happens if one exit is bare?
5. Why may `break value;` appear in `loop` but not in `while` or `for`?
6. For `for item in collection.iter()`, where do you look to determine the type and ownership behavior of `item`?
7. Define an enum `JobState` with unit, tuple, and struct variants. Construct one value of each form.
8. Write an exhaustive match over `Bool` without using `_`.
9. Why is the arm after `_` unreachable in `match value { _ => 0, 3 => 1 }`?
10. Implement `fn first_even(values: &[Int32]) -> Option<Int32>` using a `for` loop and early return.
11. Change the deployment transition case study so an invalid event produces a distinct result instead of silently preserving the current phase.
12. Review a wildcard arm in one of your own state machines. Which future variants could it accidentally absorb?

Source notes

1. Primitive control-flow grammar, statement disambiguation, enum variants, and pattern forms: `STARKLANG/docs/spec/02-Syntax-Grammar.md`.
2. Branch unification, loop result typing, iterator requirements, and never-type coercion: `STARKLANG/docs/spec/03-Type-System.md`.
3. Return-path, reachability, break validation, match exhaustiveness, and pattern type checking: `STARKLANG/docs/spec/04-Semantic-Analysis.md`.
4. `Option<T>`, `Iterator`, ranges, and required standard-library methods: `STARKLANG/docs/spec/06-Standard-Library.md`.
5. Implementation sequencing and the deliberate deferral of match-usefulness warnings: `STARKLANG/docs/PLAN.md`.